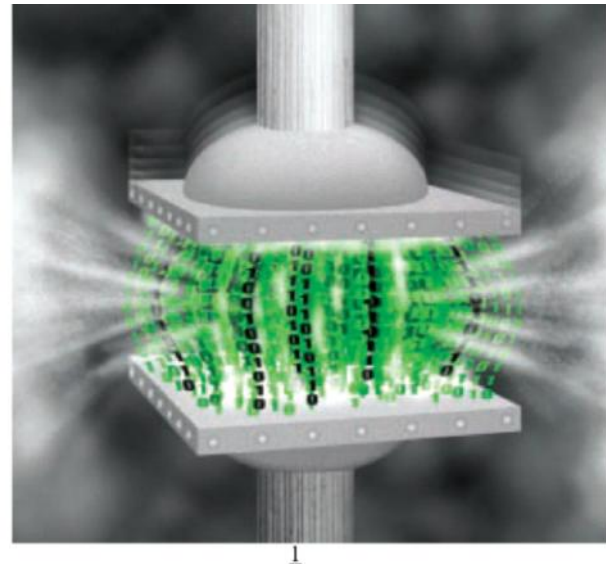
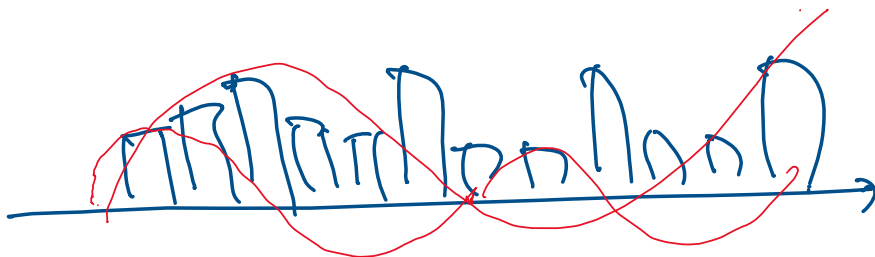

COMP 2121
DISCRETE
MATHEMATICS

Lecture 16



Weighted Trees and Prefix Codes

¹ <http://www.ams.org/mathmoments/mm41-compression.pdf>



Codeword

To represent symbols, computers use strings of 0s and 1s called **codewords**. For example, in the ASCII (American Standard Code for Information Interexchange) code, the letter A is represented by the codeword 01000001, B by 01000010 and C by 01000011. In this system each symbol is represented by some string of eight bits. Therefore, string 010000110100000101000010 is decoded as CAB.

For many purposes, this kind of representation works well. However, there are situations, as in large volume storage, where this is not an efficient method. In a fixed length representation, such as ASCII, every symbol is represented by a codeword of the same length. A more efficient approach is to use codewords of variable lengths, where the symbols used most often have shorter codewords than symbols used less frequently. For example, in normal English usage, letters E, T, O and A are used much more frequently than letters Q, J, X and Z².

² List of letters sorted with respect to their relative frequency in the English dictionary is:

E T A O I N S R H L D C U M F P G W Y B V K X J Q Z.

List of letters sorted with respect to their relative frequency in the General Fiction (English) is:

E T A O H N I S R D L U W M C G F Y P V K B J X Z Q.

Prefix Property

Suppose that the messages use only eight characters: E, T, O, A, Q, J, X and Z. A natural assignment to try is

E	0
T	1
O	01
A	11
Q	00
J	10
X	101
Z	011

Handwritten diagram illustrating the encoding of the message ZETA. The string 0110111 is shown, with brackets indicating the decoding process. The string is decoded as ZETA, with the letters Z, E, T, and A corresponding to the codewords 011, 0, 1, and 11 respectively. The string 011 is also shown separately with brackets.

Then message ZETA can be encoded as:

0110111

However, how should the string 0110111 be decoded? Should we start looking at only the first digit, or the first two, or the first three? Depending upon the number of the first digits used, the first letter could be E, O or Z. Therefore, in order to use variable length codewords, we need to select representations that permit unambiguous decoding.

A way to do this is to construct codewords so that no codeword is the first part of any other codeword. Such a set of codewords is said to have the **prefix property**.

Valid Codewords

Example 1. Consider the set of codewords, $S = \{000, 001, 01, 10, 11\}$. Does S have a prefix property?

T C B I S

The method for decoding a string of 0s and 1s into codewords that have the prefix property is to read one digit at a time until this string of digits becomes a codeword, then repeat the process, starting with the next digit, and continue until the decoding is done.

Example 2. Using the set of codewords from the Example 1, as $B = 01$, $I = 10$, $S = 11$, $T = 000$, $C = 001$ decode the string 001110000100110000.

C S

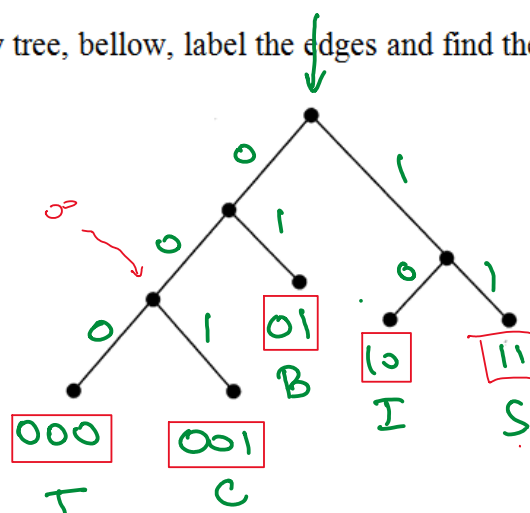
Binary Tree

Thus, the efficient method of representation should use codewords such that:

1. The codewords have the prefix property, and
2. The symbols used frequently have shorter codewords than those used less often.

Any binary tree can be used to construct a set of codewords with the prefix property, by assigning 0 to each edge from a parent to its left child, and 1 to each edge from a parent to its right child. Following the unique path from the root to a terminal vertex (leaf) will give a string of 0s and 1s.

Example 3. For the binary tree, bellow, label the edges and find the corresponding codewords.

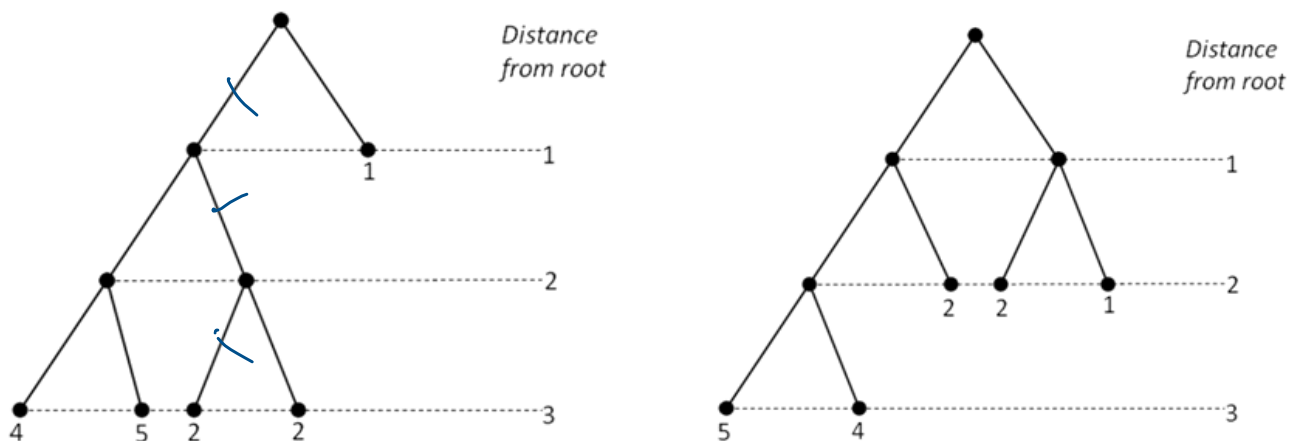


001|11|000|01|0011..
C S T ..

Weight of a tree

A **binary tree for the weights** $w_1, w_2, \dots, w_k$ is a binary tree with k leafs labeled $w_1, w_2, \dots, w_k$. A binary tree for the weights $w_1, w_2, \dots, w_k$ has **weight** $d_1w_1 + d_2w_2 + \dots + d_kw_k$, where d_i is the length of the path from the root to the vertex labeled w_i ($i=1, 2, \dots, k$).

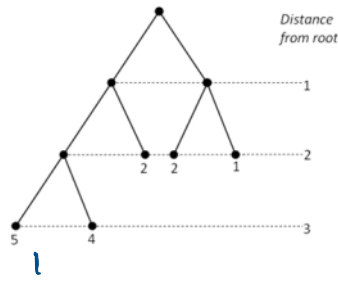
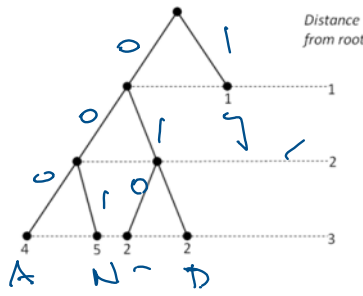
Example 4. The figure bellow shows two binary trees, both with weights 1, 2, 2, 4 and 5. What is the weight of each tree?



$$\omega(T_1) = 4(3) + 5(3) + 2(3) + 2(3) + 1(1) = 40$$

$$\omega(T_2) = 37$$

Weight of a Tree = Total Number of Bits



needs 32 bits

000 001 0001 000...

Example 5. Consider using trees from the previous example to encode the text "ANNA AND DANNY". What is the length (in bits) of the compressed text?

A A A A
N N N N N
.
D D
Y

char	freq.	cw	total # of bits
A	4/14	000	4x3 = 12
N	5/14	001	5x3 = 15
.	2/14	010	2x3 = 6
D	2/14	011	2x3 = 6
Y	1/14	1	1x1 = 1
	14		

+ = 40

Huffman's Algorithm

For the coding problem, we want to find a binary tree of the smallest possible weight, in which the weights are the frequencies of the symbols to be encoded. A binary tree is called optimal binary tree for the weights $w_1, w_2, \dots, w_k$ when its weight is as small as possible.

David A. Huffman (1925 – 1999), wrote a term paper in 1951 during his graduate work at MIT. The paper is the algorithm that constructs the optimal binary tree. Huffman coding is the fundamental algorithm in data compression. It is extensively used to compress bit strings representing text and it also plays an important role in compressing audio and image files.

The basic Algorithm:

1. Scan text to be compressed and count the occurrence of each character.
2. Sort or prioritize characters based on number of occurrences in text.
3. Build Huffman code tree based on prioritized list.
4. Perform a traversal of tree to determine all codewords.
5. Scan text again and create new file using the Huffman codes

Step 3 is explained in detail in the following algorithm.

Building the Tree

Building the tree

Create binary tree nodes with character and frequency of each character.

Place nodes in a priority queue. The lower the occurrence, the higher the priority in the queue.

While priority queue contains two or more nodes {

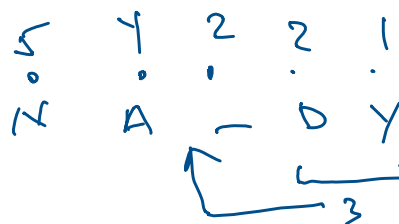
- ✓ Create new node
 - ✓ Dequeue node and make it left subtree
 - ✓ Dequeue next node and make it right subtree
 - ✓ Calculate the frequency of new node as the sum of frequencies of left and right children.
 - ✓ Enqueue new node back into queue
- }

Example 6. Build Huffman tree for the short text “ANNA AND DANNY”.

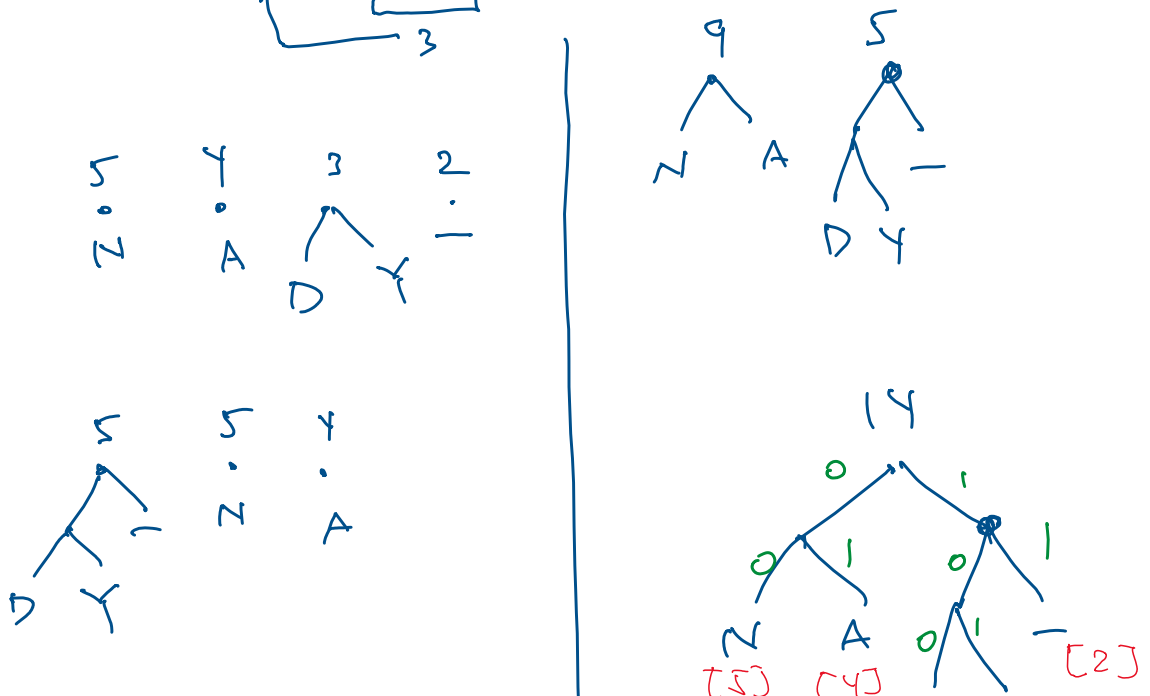
Frequency table:

Char	Frequency
A	4
N	5
space	2
D	2
Y	1

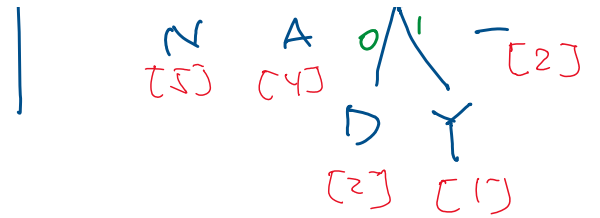
Initial Queue:



Build tree:



✓ 1



$$\omega(\tau) = 2(5) + 2(4) + 3(2) + 3(1) + 2(2) = 31$$

Lower bound for lossless compression.

Claude Shannon (1916 – 2001), “the father of information theory”, derived a lower bound for any lossless compression.

Let n be a number of characters in a file, and let p_i be the probability that any given character in the file is character i . Then, lower bound on the number of bits per character that any encoding scheme would need is:

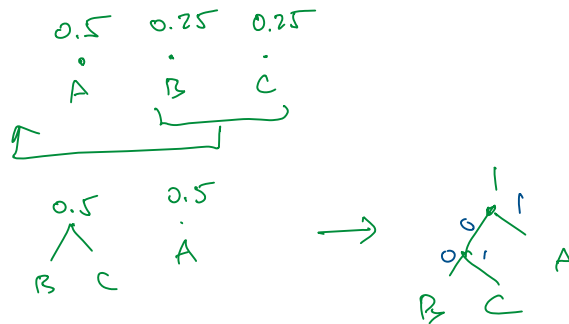
$$\sum_i^n p_i \log_2 \left(\frac{1}{p_i} \right)$$

This important property is called the entropy.

Example 7. Consider the following table with normalized frequencies (i.e. probabilities)

Char	Normalized Frequency
A	0.5
B	0.25
C	0.25

Build Huffman tree and calculate the expected number of bits per character.



TYPICAL STRING

AACB

A: 1
B: 00
C: 01

$$\frac{6 \text{ bits}}{4 \text{ char}} = 1.5 \text{ bits on average (expected)}$$

expected # of bits per char is:

$$(0.5)(1) + (0.25)(2) + (0.25)(2) = 1.5 \text{ bits/char}$$

The lower bound on the number of bits per character is:

$$\sum_i^3 p_i \log_2 \left(\frac{1}{p_i} \right) = 0.5 \cdot \log_2 \left(\frac{1}{0.5} \right) + 0.25 \cdot \log_2 \left(\frac{1}{0.25} \right) + 0.25 \cdot \log_2 \left(\frac{1}{0.25} \right) = 1.5$$

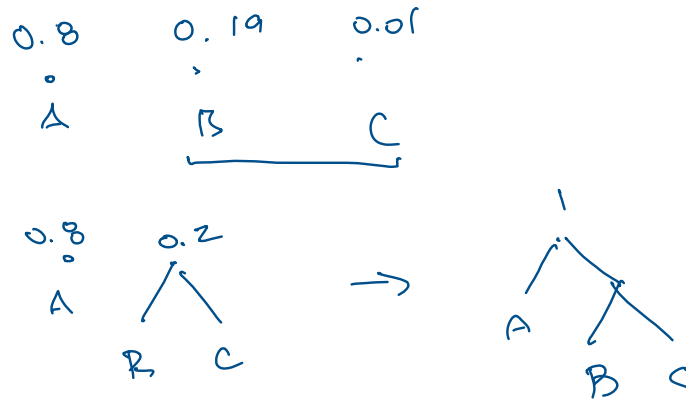


Obstacles...

Example 8. Consider the following table with normalized frequencies (i.e. probabilities)

Char	Normalized Frequency
A	0.8
B	0.19
C	0.01

Build Huffman tree and calculate the expected number of bits per character.



$$\left. \begin{array}{l} A = 0 \\ B = 10 \\ C = 11 \end{array} \right\}$$

$$(0.8)(1) + (0.19)(2) + (0.01)(2) = 1.2 \quad \begin{array}{l} \text{bits} \\ \text{char} \end{array}$$

The lower bound on the number of bits per character is:

$$\sum_i^3 p_i \log_2 \left(\frac{1}{p_i} \right) = 0.8 \cdot \log_2 \left(\frac{1}{0.8} \right) + 0.19 \cdot \log_2 \left(\frac{1}{0.19} \right) + 0.01 \cdot \log_2 \left(\frac{1}{0.01} \right) = 0.779$$

Characters... Bits... Symbols!



Encoding Multiple Characters

Example 9. Consider the following table with normalized frequencies (i.e. probabilities)

Char	Normalized Frequency
A	0.8
B	0.19
C	0.01

Handwritten sequence: A A A A B B A A B A A A C A (with a blue 'X' over the final 'A')

The minimum number of bits per character is the same as in the previous example: 0.779

$$0.9 \times 0.9 = 0.64$$

Build Huffman tree and calculate the expected number of bits per character based on the following probability distribution (obtained from the table above):

Symbol	Normalized Frequency
AA	0.6400
AB	0.1520
AC	0.0080
BA	0.1520
BB	0.0361
BC	0.0019
CA	0.0080
CB	0.0019
CC	0.0001

Handwritten: C W

Handwritten: 0

Handwritten: 11

Handwritten: 10111

Handwritten: 100

Handwritten: 1010

Handwritten: 1011011

Handwritten: 101100

Handwritten: 10110100

Handwritten: 10110101

expected # of bits

$$(0.64)(1) + (0.1520)(7) + (0.008)(5)$$

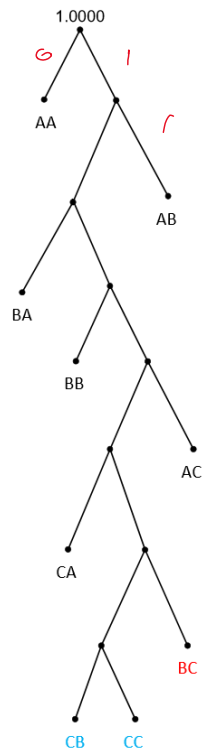
$$+ (0.1520)(3) + \dots =$$

$$= 1.6617 \frac{\text{bits}}{\text{Char}}$$

$$\text{for } \frac{1.6617}{2} = 0.83085 \frac{\text{bits}}{\text{Char}} \rightarrow$$

$$1.2$$

$$0.779$$



0.6400 • AA	0.1520 • AB	0.1520 • BA	0.0361 • BB	0.0080 • AC	0.0080 • CA	0.0019 • BC	0.0019 • CB	0.0001 • CC
-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------	-------------------

